1. Bit Stuffing

C Program

```
#include <stdio.h>

int main()
{
    char data[100], stuffed[200];
    int i, j = 0, count = 0;

    printf("Enter the bit stream: ");
    scanf("%s", data);

    for (i = 0; data[i] != '\\0'; i++)
    {
        stuffed[j++] = data[i];
        if (data[i] == '1')
            count++;
        else
            count = 0;

        if (count == 5)
        {
            stuffed[j++] = '0';
            count = 0;
        }
    }
    stuffed[j] = '\\0';

    printf("stuffed bit stream: %s\\n", stuffed);
    return 0;
}
```

Sample Output

Enter the bit stream: 1111101111110
stuffed bit stream: 111110011111010

2. Character Stuffing

C Program

```
#include <stdio.h>
#include <string.h>

int main()
{
    char start, end;
    char data[100], stuffed[300];
    int i, j = 0;

    printf("Enter input string: ");
    scanf("%s", data);

    printf("Enter starting character: ");
    scanf(" %c", &start);

    printf("Enter ending character: ");
    scanf(" %c", &end);

    stuffed[j++] = start;

    for (i = 0; i < strlen(data); i++)
    {
        if (data[i] == start || data[i] == end)
            stuffed[j++] = data[i];

        stuffed[j++] = data[i];
    }

    stuffed[j++] = end;
    stuffed[j] = '\\0';

    printf("\\nIn stuffed frame: %s\\n", stuffed);
    return 0;
}
```

Sample Output

```
Enter input string: ABCDAB
Enter starting character: A
Enter ending character: B

In stuffed frame: AAABBCDAABBB
```

3. Checksum

C Program

```
#include <stdio.h>
#include <string.h>

int main()
{
    char n1[100], n2[100], sum[100], checksum[100];
    int len, i, s, carry;

    printf("Enter the first Binary Data: ");
    scanf("%s", n1);

    printf("Enter the second Binary Data: ");
    scanf("%s", n2);

    if (strlen(n1) != strlen(n2))
    {
        printf("Error: Both binary numbers must be of the same length.\\n");
        return 0;
    }

    len = strlen(n1);
    carry = 0;

    for (i = len - 1; i >= 0; i--)
    {
        s = (n1[i] - '0') + (n2[i] - '0') + carry;
        carry = s / 2;
        sum[i] = (s % 2) + '0';
    }
    sum[len] = '\\0';

    if (carry > 0)
    {
        for (i = len - 1; i >= 0; i--)
        {
            s = (sum[i] - '0') + carry;
            carry = s / 2;
            sum[i] = (s % 2) + '0';
        }
    }

    printf("\\nSum of Data: %s\\n", sum);

    for (i = 0; i < len; i++)
        checksum[i] = (sum[i] == '0') ? '1' : '0';

    checksum[len] = '\\0';
    printf("Generated checksum: %s\\n", checksum);
    return 0;
}
```

Sample Output

```
Enter the first Binary Data: 10101010
Enter the second Binary Data: 11001100

Sum of Data: 01110111
Generated checksum: 10001000
```

4. Hamming Code

C Program

```
#include <stdio.h>

int main()
{
    int data[8], received[8];
    int d3, d5, d6, d7;
    int p1, p2, p4;
    int i, error_pos = 0;

    printf("Enter 4 data bits (0 or 1):\\n");
    printf("Enter data bit d3: "); scanf("%d", &d3);
    printf("Enter data bit d5: "); scanf("%d", &d5);
    printf("Enter data bit d6: "); scanf("%d", &d6);
    printf("Enter data bit d7: "); scanf("%d", &d7);

    p1 = d3 ^ d5 ^ d7;
    p2 = d3 ^ d6 ^ d7;
    p4 = d5 ^ d6 ^ d7;

    data[1] = p1; data[2] = p2; data[3] = d3;
    data[4] = p4; data[5] = d5; data[6] = d6; data[7] = d7;

    printf("\\nGenerated Hamming Code: ");
    for (i = 1; i <= 7; i++)
        printf("%d ", data[i]);
    printf("\\n");

    printf("\\nEnter the received 7-bit Hamming Code:\\n");
    for (i = 1; i <= 7; i++)
    {
        printf("Enter bit at position %d: ", i);
        scanf("%d", &received[i]);
    }

    for (i = 1; i <= 7; i++)
    {
        if (data[i] != received[i])
        {
            error_pos = i;
            break;
        }
    }

    if (error_pos == 0)
        printf("\\nResult: Received code matches the generated code perfectly. No error!\\n");
    else
    {
        printf("\\nResult: Mismatch detected! Position %d is not equal.\\n", error_pos);
        received[error_pos] = data[error_pos];

        printf("Corrected Received Code: ");
        for (i = 1; i <= 7; i++)
            printf("%d ", received[i]);
        printf("\\n");
    }
    return 0;
}
```

Sample Output

```
Enter 4 data bits (0 or 1):
Enter data bit d3: 1
Enter data bit d5: 0
Enter data bit d6: 1
Enter data bit d7: 1

Generated Hamming Code: 0 1 1 0 0 1 1

Enter the received 7-bit Hamming Code:
Enter bit at position 1: 0
Enter bit at position 2: 1
Enter bit at position 3: 1
Enter bit at position 4: 0
Enter bit at position 5: 1
Enter bit at position 6: 1
Enter bit at position 7: 1

Result: Mismatch detected! Position 5 is not equal.
Corrected Received Code: 0 1 1 0 0 1 1
```

5. Go-Back-N Sliding Window Protocol

C Program

```
#include <stdio.h>

int main()
{
    int total, window, i = 1, j, ack;

    printf("Enter total frames: ");
    scanf("%d", &total);
    printf("Enter window size: ");
    scanf("%d", &window);

    while (i <= total)
    {
        printf("\\nSent: ");
        for (j = i; j < i + window && j <= total; j++)
            printf("%d ", j);

        printf("\\nEnter Last ACK: ");
        scanf("%d", &ack);

        if (ack >= i && ack < j)
        {
            printf("ACK %d OK. Moving window.\\n", ack);
            i = ack + 1;
        }
        else
            printf("Error! Retransmitting from %d.\\n", i);
    }

    printf("\\nAll Success!\\n");
    return 0;
}
```

Sample Output

```
Enter total frames: 5
Enter window size: 3

Sent: 1 2 3
Enter Last ACK: 3
ACK 3 OK. Moving window.

Sent: 4 5
Enter Last ACK: 5
ACK 5 OK. Moving window.

All Success!
```

6. Selective Repeat Sliding Window Protocol (as supplied)

C Program

```
#include <stdio.h>

int main()
{
    int total, window, i = 1, j, choice, lost_frame;

    printf("Enter total frames: ");
    scanf("%d", &total);
    printf("Enter window size: ");
    scanf("%d", &window);

    while (i <= total)
    {
        printf("\\nSent: ");
        for (j = i; j < i + window && j <= total; j++)
            printf("%d ", j);

        printf("\\nAny frame lost? (1=Yes, 0=No): ");
        scanf("%d", &choice);

        if (choice == 1)
        {
            printf("Enter lost frame: ");
            scanf("%d", &lost_frame);

            if (lost_frame >= i && lost_frame < j)
            {
                printf("Retransmitting only frame %d...\\n", lost_frame);
                printf("ACK %d OK. Moving window.\\n", j - 1);
                i = j;
            }
            else
                printf("Invalid frame number!\\n");
        }
        else
        {
            printf("ACK %d OK. Moving window.\\n", j - 1);
            i = j;
        }
    }

    printf("\\nAll Success!\\n");
    return 0;
}
```

Sample Output

```
Enter total frames: 5
Enter window size: 3

Sent: 1 2 3
Any frame lost? (1=Yes, 0=No): 1
Enter lost frame: 2
Retransmitting only frame 2...
ACK 3 OK. Moving window.

Sent: 4 5
Any frame lost? (1=Yes, 0=No): 0
ACK 5 OK. Moving window.

All Success!
```

7. Stop-and-Wait Protocol with Timeout (as supplied)

C Program

```
#include <stdio.h>
#include <unistd.h>
#include <time.h>

#define TIMEOUT 5

int main()
{
    int total, i = 1, choice, t;
    time_t start;

    printf("Total frames: ");
    scanf("%d", &total);

    while (i <= total)
    {
        printf("\\nSent Frame: %d\\nACK? (1=OK, 0=Retransmit, or wait %ds for Timeout): ",
               i, TIMEOUT);

        start = time(NULL);
        choice = -1;

        t = sleep(2);
        scanf("%d", &choice);

        if (choice == 1)
            printf("ACK %d OK. Moving forward.\\n", i++);
        else if (choice == 0)
            printf("ACK 0 received! Retransmitting.\\n");
        else if (t == sleep(2))
            printf("Timeout! No input given. Retransmitting.\\n");
    }

    printf("\\nSuccess! All frames sent.\\n");
    return 0;
}
```

Sample Output

```
Total frames: 3

Sent Frame: 1
ACK? (1=OK, 0=Retransmit, or wait 5s for Timeout): 1
ACK 1 OK. Moving forward.

Sent Frame: 2
ACK? (1=OK, 0=Retransmit, or wait 5s for Timeout): 1
ACK 2 OK. Moving forward.

Sent Frame: 3
ACK? (1=OK, 0=Retransmit, or wait 5s for Timeout): 1
ACK 3 OK. Moving forward.

Success! All frames sent.
```

8. Distance Vector Routing Algorithm

C Program

```
#include <stdio.h>

int main()
{
    int n, i, j, k, changed = 1;
    int d[10][10];

    printf("Nodes: ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 10)
        return 1;

    printf("Matrix:\\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &d[i][j]);

    while (changed)
    {
        changed = 0;
        for (i = 0; i < n; i++)
            for (j = 0; j < n; j++)
                for (k = 0; k < n; k++)
                    if (d[i][k] != 9999 && d[k][j] != 9999 &&
                        d[i][j] > d[i][k] + d[k][j])
                    {
                        d[i][j] = d[i][k] + d[k][j];
                        changed = 1;
                    }
    }

    printf("\\nResult:\\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
            printf("%d ", d[i][j]);
        printf("\\n");
    }
    return 0;
}
```

Sample Output

Nodes: 3
Matrix:
0 2 7
2 0 1
7 1 0

Result:
0 2 3
2 0 1
3 1 0

9. Leaky Bucket Algorithm

C Program

```
#include <stdio.h>

int main() {
    // Variables sorted alphabetically
    int bucket_size, leak_rate, n;
    int current_storage = 0, incoming;

    // 1. Take configuration inputs from the user
    printf("Enter bucket capacity, leak rate, and number of inputs: ");
    scanf("%d %d %d", &bucket_size, &leak_rate, &n);

    // 2. Process each incoming packet loop
    while (n > 0) {
        printf("\\nEnter incoming packet size: ");
        scanf("%d", &incoming);

        // Check if the packet fits in the remaining bucket space
        if (incoming <= (bucket_size - current_storage)) {
            current_storage += incoming;
            printf("Packet accepted. Current buffer: %d/%d\\n",
                   current_storage, bucket_size);
        } else {
            printf("Bucket Overflow! Dropped packet of size: %d\\n", incoming);
        }

        // The bucket leaks data at a fixed rate
        if (current_storage >= leak_rate) {
            current_storage -= leak_rate;
        } else {
            current_storage = 0; // Bucket becomes empty
        }

        printf("After leak, packets left in buffer: %d/%d\\n",
               current_storage, bucket_size);
        n--;
    }

    return 0;
}
```

Sample Output

```
Enter bucket capacity, leak rate, and number of inputs: 10 1 4

Enter incoming packet size: 8
Packet accepted. Current buffer: 8/10
After leak, packets left in buffer: 7/10

Enter incoming packet size: 5
Bucket Overflow! Dropped packet of size: 5
After leak, packets left in buffer: 6/10

Enter incoming packet size: 4
Packet accepted. Current buffer: 10/10
After leak, packets left in buffer: 9/10

Enter incoming packet size: 2
Bucket Overflow! Dropped packet of size: 2
After leak, packets left in buffer: 8/10
```